Substituting a fresh variable and back recovers the term: the two-step inversion of substitution

The Open Logic Project contributors

openlogic:lambda-calculus/syntax:local-snapshot-20261013; CC BY 4.0

Editorial note. All delivered mathematical text is the author's own, copied from the pinned Open Logic Project unit file sources/olp-lambda-calculus-syntax/pinned-original-source.tex; no statement or proof step was rewritten, reordered or removed.

Editorial note. The counted proof is the author's complete argument: both compound cases are verbatim three-step calculations; the author's exercise marker stands on the base variable case alone, where the freshness hypotheses make each branch definitional.

Editorial note. The label thm:inv names the counted statement, and the proof environment immediately following it in the source is delivered verbatim as its proof.

Editorial note. The counted statement is carried by the author's own thm environment in the unit source.

Allowed standard prerequisites (editorial).

- Definition of capture-avoiding substitution. Same unit, Definition of substitution with the application and abstraction clauses. Distributes both substitutions through the term structure in the two written cases.
- Free variables. Syntax unit, Definition of FV. Reads the freshness hypothesis y not in FV(M) through the binder case.

Theorem 0.1. *If $M[y/x]$ is defined and $y \notin \mathrm{FV}\, M$, then $M[y/x][x/y] = M$.*

Proof. By induction on the formation of M.

1. M is a variable z: Exercise.
2. M is of the form (PQ). Then:

$$\begin{aligned}(PQ)[y/x][x/y] &= ((P[y/x])(Q[y/x]))[x/y] \\ &= (P[y/x][x/y])(Q[y/x][x/y]) \\ &= (PQ) \text{ by inductive hypothesis}\end{aligned}$$

3. M is of the form $\lambda[z][N]$. Because $\lambda[z][N][y/x]$ is defined, we know that $z \neq y$. So:

$$\begin{aligned}(\lambda[z][N])[y/x][x/y] & \\ &= (\lambda[z][N[y/x]])[x/y] \\ &= \lambda[z][N[y/x][x/y]] \\ &= \lambda[z][N] \text{ by inductive hypothesis}\end{aligned}$$

□